

Unidad 5: Acceso a estructura NoSQL

Clase 10 – Acceso a datos NoSQL desde aplicaciones

Objetivo

Acceder y manipular datos almacenados en estructuras NoSQL desde aplicaciones, utilizando herramientas de conectividad y frameworks de persistencia, y evaluando los resultados obtenidos.

Tema

Acceso a estructuras NoSQL desde aplicaciones.

Acceso a datos NoSQL desde aplicaciones con Python

Introducción conceptual

Las bases de datos NoSQL adquieren verdadero valor cuando dejan de ser estructuras aisladas y comienzan a formar parte de una solución de software. Una colección documental en MongoDB, una clave temporal en Redis o una tabla distribuida en Cassandra no resuelven por sí mismas un problema de negocio. Su utilidad aparece cuando una aplicación puede conectarse a esas estructuras, enviar operaciones, interpretar respuestas y tomar decisiones a partir de los datos obtenidos.

El acceso a datos desde aplicaciones permite pasar del uso manual de comandos a un escenario más cercano a los sistemas reales. En una organización, los usuarios no interactúan directamente con la consola de MongoDB o Redis. Utilizan una aplicación web, móvil o de escritorio que recibe solicitudes, valida información, consulta una base, registra eventos y devuelve una respuesta. Por ese motivo, comprender cómo una aplicación se conecta a una base NoSQL es un paso fundamental para diseñar soluciones modernas de datos.

El eje de este apunte es el acceso a estructuras NoSQL desde aplicaciones utilizando Python como lenguaje de programación. El desarrollo se apoya en un caso de ejemplo de gestión de turnos médicos, donde MongoDB se utiliza para almacenar información persistente y Redis se utiliza para manejar estados rápidos y temporales. Esta combinación permite comprender un principio importante de la ingeniería de datos actual: no todas las bases cumplen la misma función dentro de una arquitectura.

Del almacenamiento aislado al acceso desde aplicaciones

Una base de datos puede ser utilizada de diferentes maneras. En un entorno de aprendizaje inicial, es habitual ejecutar comandos directamente sobre el motor de base de datos. Por ejemplo, se puede abrir una consola de MongoDB e insertar un

documento, consultar una colección o actualizar un campo. Esa práctica es útil para comprender el modelo de datos, pero no representa completamente el funcionamiento de un sistema real.

En una aplicación real, el acceso a datos ocurre mediante una secuencia ordenada. Un usuario solicita una operación, la aplicación recibe esa solicitud, valida los datos recibidos, establece comunicación con una base de datos, ejecuta una operación, analiza el resultado y responde. El acceso a datos no consiste solamente en “guardar” o “leer” información, sino en integrar la base dentro de un proceso lógico.

En un sistema de turnos médicos, por ejemplo, una persona puede solicitar un turno con un profesional. La aplicación debe verificar si el turno existe, si está disponible, si otro paciente no lo tomó antes, si se puede bloquear temporalmente mientras se confirma la reserva y si finalmente corresponde registrar la operación. Cada paso implica decisiones de acceso a datos.

La diferencia entre usar una base desde consola y usarla desde una aplicación puede resumirse así: la consola permite ejecutar operaciones directas; la aplicación organiza esas operaciones dentro de reglas, validaciones, flujos y respuestas orientadas a un usuario o proceso de negocio.

Aplicación, base de datos y capa de conexión

Una aplicación no se comunica con una base de datos de forma automática. Para hacerlo necesita una capa de conexión. Esta capa está compuesta por librerías, drivers o frameworks que permiten traducir instrucciones del lenguaje de programación en operaciones comprensibles para la base de datos.

Un driver es una biblioteca de software que permite que un lenguaje de programación se conecte con una base de datos específica. En Python, por ejemplo, la librería pymongo permite conectarse con MongoDB, mientras que la librería redis permite conectarse con Redis. Sin estos componentes, Python no sabría cómo enviar una consulta a MongoDB ni cómo crear una clave en Redis.

La conexión cumple una función similar a un canal de comunicación. La aplicación indica a qué servidor desea conectarse, con qué puerto, con qué credenciales si corresponde, y sobre qué base o estructura desea operar. Una vez establecida la conexión, puede ejecutar operaciones como insertar, consultar, actualizar o eliminar datos.

Es importante diferenciar tres conceptos que suelen confundirse:

- La aplicación es el programa que contiene la lógica de negocio.
- El driver es la librería que permite comunicarse con la base.
- La base de datos es el sistema que almacena, organiza y responde operaciones sobre los datos.

En el caso de turnos médicos, la aplicación podría estar escrita en Python. Esa aplicación usa pymongo para registrar reservas en MongoDB y usa redis para bloquear temporalmente un turno mientras se confirma la operación. MongoDB y Redis no reemplazan a la aplicación: son recursos de datos utilizados por ella.

Operaciones CRUD y acceso programático

CRUD es una sigla formada por las palabras Create, Read, Update y Delete. Representa las cuatro operaciones básicas que una aplicación suele realizar sobre los datos: crear, leer, actualizar y eliminar.

En una base documental como MongoDB, estas operaciones se aplican sobre documentos. Un documento es una estructura flexible compuesta por pares de campo y valor. En Python, un documento puede representarse fácilmente como un diccionario, lo que facilita la comprensión del modelo.

Por ejemplo, una reserva de turno médico podría representarse así:

```
{
  "paciente_id": "pac_1001",
  "medico_id": "med_2001",
  "especialidad": "Cardiología",
  "fecha": "2026-05-20",
  "hora": "10:30",
  "estado": "confirmado"
}
```

Esta estructura no exige una tabla con columnas fijas como ocurre en un modelo relacional tradicional. MongoDB permite almacenar documentos con campos flexibles, aunque esa flexibilidad debe ser administrada con criterio. Una aplicación no debería insertar documentos desordenados o incoherentes solo porque la base lo permite.

En Redis, el enfoque es diferente. Redis es una base clave-valor en memoria, diseñada para operaciones rápidas. En lugar de documentos complejos orientados al historial, suele trabajar con claves que representan estados, contadores, bloqueos o estructuras temporales.

Por ejemplo, para bloquear un turno mientras un paciente confirma la reserva, se podría utilizar una clave como:

```
turno:bloqueado:med_2001:2026-05-20:10:30
```

Esa clave puede existir durante algunos minutos. Si la clave existe, significa que el turno está siendo procesado por otro usuario. Si no existe, la aplicación puede intentar

bloquearlo. Este uso es muy distinto del almacenamiento persistente de una reserva confirmada en MongoDB.

Persistencia, estado temporal y responsabilidad de cada base

Una decisión fundamental en arquitecturas NoSQL es definir qué responsabilidad cumple cada base de datos. No todas las necesidades de datos tienen la misma naturaleza. Algunas requieren persistencia, trazabilidad y consulta posterior. Otras requieren velocidad, disponibilidad inmediata o manejo de estados temporales.

La persistencia se refiere a la capacidad de conservar datos más allá de la ejecución inmediata de una operación. Una reserva confirmada, el historial de turnos de un paciente o los datos de un médico deben quedar almacenados para futuras consultas. MongoDB resulta adecuado para ese tipo de información porque permite guardar documentos estructurados y consultarlos luego por diferentes criterios.

El estado temporal representa información válida solo durante un período breve. Un bloqueo momentáneo de un turno, un contador de intentos, una sesión de usuario o una clave con vencimiento no necesariamente deben permanecer como parte del historial principal. Redis es adecuado para este tipo de necesidad porque permite trabajar en memoria y asignar tiempos de expiración mediante TTL.

TTL significa Time To Live, es decir, “tiempo de vida”. En Redis, una clave puede crearse con un vencimiento automático. Cuando se cumple ese tiempo, la clave desaparece. Esto es útil para procesos donde un dato solo tiene sentido durante unos segundos o minutos.

En un sistema de turnos médicos, la diferencia puede expresarse de manera precisa:

- MongoDB registra la reserva confirmada porque esa información forma parte del historial del sistema.
- Redis bloquea temporalmente un turno porque esa información solo sirve mientras se completa la operación.

Un error frecuente consiste en usar una sola base para todo sin distinguir la naturaleza de cada dato. Guardar todos los estados temporales en MongoDB puede generar consultas innecesarias y mayor complejidad. Guardar todo el historial en Redis puede ser riesgoso si no se configuró adecuadamente la persistencia y si el modelo no fue diseñado para consulta histórica. La elección de la base debe responder al uso esperado del dato.

Python como lenguaje de integración

Python es un lenguaje de programación ampliamente utilizado en análisis de datos, automatización, desarrollo de aplicaciones, inteligencia artificial e integración de sistemas. Su sintaxis clara y su ecosistema de librerías lo convierten en una herramienta adecuada para trabajar con bases NoSQL desde aplicaciones.

En el contexto de acceso a datos, Python permite escribir scripts que se conectan a diferentes motores, ejecutan operaciones y procesan resultados. Un script es un archivo de código que puede ejecutarse para realizar una tarea específica. En este caso, los scripts permitirán probar la conexión con MongoDB y Redis, insertar datos, consultar información y simular una reserva de turno.

Python no reemplaza a la base de datos. Su función es actuar como capa de aplicación. La base almacena o administra datos; Python decide cuándo conectarse, qué operación ejecutar, cómo interpretar la respuesta y qué acción tomar después.

El uso de Python también permite comprender el concepto de persistencia políglota. La persistencia políglota consiste en utilizar más de un tipo de base de datos dentro de una misma solución, asignando a cada una la responsabilidad que mejor se ajusta a su modelo. En el ejemplo de turnos médicos, MongoDB y Redis trabajan juntas, pero no hacen lo mismo.

Preparación del entorno de trabajo

Para trabajar con acceso a bases NoSQL desde Python se necesita preparar un entorno mínimo. El entorno está compuesto por el lenguaje de programación, las herramientas de edición, las bases de datos y las librerías de conexión.

El objetivo de esta preparación es poder ejecutar código Python que se conecte con MongoDB y Redis. No se requiere desarrollar una aplicación web completa ni instalar frameworks complejos. El foco está en comprender la comunicación entre una aplicación simple y las bases NoSQL.

Instalación de Python

Python puede descargarse desde el sitio oficial del lenguaje. Durante la instalación en Windows, es importante marcar la opción "Add Python to PATH". PATH es una variable del sistema operativo que permite ejecutar programas desde la terminal sin tener que indicar manualmente su ubicación completa. Si Python no se agrega al PATH, puede ocurrir que el sistema no reconozca el comando python o pip.

Después de instalar Python, se debe abrir una terminal. En Windows puede usarse PowerShell, Símbolo del sistema o la terminal integrada de Visual Studio Code.

Para verificar que Python está instalado, se ejecuta:

```
python --version
```

En algunos equipos, especialmente si hay más de una versión instalada, puede ser necesario ejecutar:

```
py --version
```

Si la instalación fue correcta, la terminal mostrará una versión de Python, por ejemplo:

```
Python 3.12.x
```

También se debe verificar que pip esté disponible. Pip es el gestor de paquetes de Python. Sirve para instalar librerías externas, como pymongo y redis.

```
pip --version
```

Si el comando no funciona, puede probarse:

```
python -m pip --version
```

La forma python -m pip suele ser más segura porque le indica explícitamente a Python que ejecute el módulo pip asociado a esa instalación.

Instalación de Visual Studio Code

Visual Studio Code es un editor de código que permite escribir, ejecutar y organizar archivos de programación. No es obligatorio, pero facilita mucho el trabajo porque incluye terminal integrada, resaltado de sintaxis y extensiones para Python.

Después de instalar Visual Studio Code, conviene agregar la extensión oficial de Python. Esta extensión permite reconocer archivos .py, seleccionar el intérprete de Python y ejecutar scripts con mayor comodidad.

Un archivo Python tiene extensión .py. Por ejemplo:

```
conexion_mongodb.py
```

El código escrito en ese archivo puede ejecutarse desde la terminal ubicada en la misma carpeta mediante:

```
python conexion_mongodb.py
```

Creación de una carpeta de trabajo

Es recomendable crear una carpeta específica para los archivos del proyecto. Por ejemplo:

```
nosql_python_turnos
```

Dentro de esa carpeta se pueden guardar los scripts de conexión, las pruebas con MongoDB, las pruebas con Redis y el ejemplo integrado. Mantener los archivos ordenados evita confusiones al ejecutar comandos desde la terminal.

Una estructura simple podría ser:

```
nosql_python_turnos/
```

```
    conexion_mongodb.py
```

```
    conexion_redis.py
```

```
    caso_turnos.py
```

La terminal debe ubicarse dentro de esa carpeta antes de ejecutar los scripts. En Windows, el comando cd permite cambiar de carpeta. Por ejemplo:

```
cd C:\ruta\a\nosql_python_turnos
```

Uso de entornos virtuales en Python

Un entorno virtual es una carpeta especial que permite instalar librerías para un proyecto sin afectar otras instalaciones de Python. Su uso es una buena práctica porque evita conflictos entre versiones de paquetes.

Para crear un entorno virtual dentro de la carpeta del proyecto, se ejecuta:

```
python -m venv venv
```

El primer venv indica el módulo que crea entornos virtuales. El segundo venv es el nombre de la carpeta que se creará.

Luego se activa el entorno virtual. En Windows PowerShell:

```
.\venv\Scripts\Activate.ps1
```

En Símbolo del sistema:

```
venv\Scripts\activate
```

Cuando el entorno está activo, suele aparecer el nombre (venv) al inicio de la línea de la terminal. Esto indica que las librerías que se instalen quedarán asociadas a ese proyecto.

Si PowerShell no permite activar el entorno por restricciones de ejecución, puede usarse esta alternativa desde la misma terminal:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Luego se vuelve a ejecutar:

```
.\venv\Scripts\Activate.ps1
```

Esta modificación se aplica solo al proceso actual de la terminal y permite activar el entorno virtual sin cambiar permanentemente la configuración del sistema.

Instalación de librerías de conexión

Con el entorno virtual activado, se instalan las librerías necesarias.

Para conectarse a MongoDB desde Python se utiliza pymongo:

```
pip install pymongo
```

Para conectarse a Redis desde Python se utiliza redis:

```
pip install redis
```

También puede instalarse python-dotenv si se desea trabajar con variables de entorno, aunque para una práctica inicial no es imprescindible:

```
pip install python-dotenv
```

Una variable de entorno permite guardar configuraciones fuera del código, como cadenas de conexión o contraseñas. Esto es importante en proyectos reales porque evita escribir credenciales sensibles directamente dentro de los scripts.

Para verificar qué paquetes están instalados en el entorno actual, se puede ejecutar:

```
pip list
```

Instalación y uso de MongoDB

MongoDB es una base de datos documental. Organiza la información en bases de datos, colecciones y documentos. Una colección puede entenderse como un conjunto de documentos relacionados. No es exactamente igual a una tabla relacional porque no exige una estructura fija de columnas, aunque en la práctica conviene mantener coherencia entre documentos de una misma colección.

Existen varias formas de usar MongoDB. Para una práctica local, se puede instalar MongoDB Community Server en el equipo. También puede usarse MongoDB Atlas, que es la versión administrada en la nube. Para un primer contacto, la instalación local permite trabajar sin depender de una cuenta externa.

Después de instalar MongoDB Community Server, el servicio suele quedar disponible en el puerto 27017. Un puerto es un punto de comunicación utilizado por una aplicación para conectarse con otro servicio. En MongoDB local, la dirección típica es:

```
mongodb://localhost:27017/
```

localhost representa la propia computadora. El número 27017 es el puerto por defecto de MongoDB.

También es recomendable instalar MongoDB Compass. MongoDB Compass es una herramienta gráfica que permite ver bases, colecciones y documentos sin usar solo comandos. No reemplaza el acceso desde Python, pero ayuda a verificar si los documentos fueron insertados correctamente.

Prueba de conexión con MongoDB desde Python

El primer paso consiste en crear un archivo llamado `conexion_mongodb.py` dentro de la carpeta de trabajo. El objetivo de este script es comprobar que Python puede comunicarse con MongoDB.

```
from pymongo import MongoClient
```

```
cliente = MongoClient("mongodb://localhost:27017/")
```

```
base = cliente["clinica_turnos"]
```

```
coleccion = base["turnos"]
```



```
turno = {
    "paciente_id": "pac_1001",
    "medico_id": "med_2001",
    "especialidad": "Cardiología",
    "fecha": "2026-05-20",
    "hora": "10:30",
    "estado": "disponible"
}
```

```
resultado = coleccion.insert_one(turno)
```

```
print("Documento insertado con ID:", resultado.inserted_id)
```

Este código realiza varias acciones. Primero importa MongoClient, que es el objeto de conexión provisto por pymongo. Luego crea un cliente conectado a MongoDB local. Después selecciona una base llamada clinica_turnos y una colección llamada turnos. Si la base o la colección no existen, MongoDB las crea al insertar el primer documento.

El documento turno se define como un diccionario de Python. La operación insert_one inserta un único documento en la colección. El resultado contiene el identificador generado por MongoDB.

Para ejecutar el archivo desde la terminal:

```
python conexion_mongodb.py
```

Si todo funciona correctamente, aparecerá un mensaje similar a:

Documento insertado con ID: 665...

Luego se puede abrir MongoDB Compass, conectarse a mongodb://localhost:27017/ y verificar la existencia de la base clinica_turnos, la colección turnos y el documento insertado.

Consulta de documentos en MongoDB desde Python

Leer datos desde una aplicación implica ejecutar una consulta y recorrer los resultados. En MongoDB, find permite recuperar documentos que cumplen una condición.

```
from pymongo import MongoClient
```

```
cliente = MongoClient("mongodb://localhost:27017/")
base = cliente["clinica_turnos"]
coleccion = base["turnos"]

turnos_cardiologia = coleccion.find({"especialidad": "Cardiología"})
```

```
for turno in turnos_cardiologia:
```

```
    print(turno)
```

La consulta busca documentos donde el campo especialidad sea igual a Cardiología. El resultado puede contener uno o muchos documentos. Por eso se recorre con un ciclo for.

Este ejemplo permite observar una diferencia importante entre consultar desde la consola y consultar desde una aplicación. En la aplicación, el resultado puede ser utilizado para tomar decisiones. No solo se imprime; también podría enviarse a una interfaz, validarse, transformarse o combinarse con otros datos.

Actualización de documentos en MongoDB desde Python

Actualizar datos significa modificar documentos existentes. En MongoDB, la operación update_one actualiza el primer documento que coincide con un filtro.

```
from pymongo import MongoClient
```

```
cliente = MongoClient("mongodb://localhost:27017/")
```

```
base = cliente["clinica_turnos"]
```

```
coleccion = base["turnos"]
```

```
filtro = {
```

```
    "medico_id": "med_2001",
```

```
    "fecha": "2026-05-20",
```

```
    "hora": "10:30"
```

```
}
```

```
nuevos_valores = {
```

```
"$set": {"estado": "confirmado"}
}
```

```
resultado = coleccion.update_one(filtro, nuevos_valores)
```

```
print("Documentos encontrados:", resultado.matched_count)
```

```
print("Documentos modificados:", resultado.modified_count)
```

El filtro indica qué documento se desea encontrar. El operador \$set modifica el valor de un campo sin reemplazar todo el documento. El resultado de la operación permite evaluar si la actualización tuvo efecto.

Esta evaluación es fundamental. Una aplicación no debería asumir que una operación funcionó solo porque no generó un error. Si matched_count es cero, significa que no se encontró ningún documento con ese filtro. Si modified_count es cero, puede significar que el documento ya tenía el valor indicado o que no se realizó ningún cambio efectivo.

Eliminación de documentos en MongoDB desde Python

Eliminar datos debe realizarse con criterio porque puede afectar la integridad del sistema. En muchos sistemas reales, en lugar de eliminar físicamente un documento, se cambia su estado. Por ejemplo, una reserva puede pasar de confirmada a cancelada sin borrar el registro.

La eliminación física se puede realizar con delete_one:

```
from pymongo import MongoClient
```

```
cliente = MongoClient("mongodb://localhost:27017/")
```

```
base = cliente["clinica_turnos"]
```

```
coleccion = base["turnos"]
```

```
filtro = {
```

```
    "paciente_id": "pac_1001",
```

```
    "medico_id": "med_2001",
```

```
    "fecha": "2026-05-20",
```

```
    "hora": "10:30"
```

```
}
```

```
resultado = coleccion.delete_one(filtro)
```

```
print("Documentos eliminados:", resultado.deleted_count)
```

En el caso de turnos médicos, suele ser más conveniente conservar trazabilidad. Una cancelación puede ser relevante para auditoría, estadísticas, disponibilidad y experiencia del paciente. Por eso, una solución más adecuada podría ser:

```
coleccion.update_one(filtro, {"$set": {"estado": "cancelado"}})
```

Este criterio muestra que el acceso a datos no es solo una cuestión técnica. También implica decisiones de diseño, trazabilidad y control.

Instalación y uso de Redis

Redis es una base de datos clave-valor en memoria. Se utiliza con frecuencia para caché, sesiones, contadores, colas livianas, rankings, bloqueos temporales y datos que requieren respuesta rápida.

Una clave en Redis identifica un valor o una estructura. Por ejemplo:

```
turno:bloqueado:med_2001:2026-05-20:10:30
```

El diseño de nombres de claves es importante. Una clave clara permite comprender qué representa el dato. En sistemas reales se suelen usar prefijos separados por dos puntos para ordenar semánticamente la información.

Redis puede instalarse de diferentes maneras. En Windows, una opción práctica es usar Docker Desktop. Docker permite ejecutar servicios en contenedores. Un contenedor es un entorno aislado que contiene una aplicación y sus dependencias. Usar Redis en Docker evita instalaciones complejas y permite iniciar o detener el servicio fácilmente.

Una vez instalado Docker Desktop, se puede abrir una terminal y ejecutar:

```
docker run --name redis-turnos -p 6379:6379 -d redis
```

Este comando descarga y ejecuta Redis en un contenedor. El parámetro `--name redis-turnos` asigna un nombre al contenedor. El parámetro `-p 6379:6379` conecta el puerto local 6379 con el puerto interno del contenedor. El parámetro `-d` ejecuta el contenedor en segundo plano. La palabra `redis` indica la imagen que se utilizará.

Para verificar que el contenedor está ejecutándose:

```
docker ps
```

Si Redis aparece en la lista, el servicio está activo.

Para detener el contenedor:

```
docker stop redis-turnos
```

Para volver a iniciarlo:

```
docker start redis-turnos
```

Redis utiliza por defecto el puerto 6379. La conexión local desde Python se realizará contra localhost y ese puerto.

Prueba de conexión con Redis desde Python

Dentro de la carpeta de trabajo, se puede crear un archivo llamado `conexion_redis.py`.

```
import redis
```

```
cliente_redis = redis.Redis(  
    host="localhost",  
    port=6379,  
    decode_responses=True  
)
```

```
cliente_redis.set("prueba:conexion", "Redis funcionando")
```

```
valor = cliente_redis.get("prueba:conexion")
```

```
print(valor)
```

La opción `decode_responses=True` permite que Redis devuelva cadenas de texto en lugar de bytes. Esto facilita la lectura de los resultados en ejemplos iniciales.

Para ejecutar el script:

```
python conexion_redis.py
```

Si la conexión funciona, la terminal mostrará:

```
Redis funcionando
```

Uso de TTL en Redis

El TTL permite que una clave expire automáticamente después de un tiempo definido. Esto es especialmente útil para bloquear temporalmente un turno mientras un paciente confirma la reserva.

```
import redis
```

```
cliente_redis = redis.Redis(
    host="localhost",
    port=6379,
    decode_responses=True
)
```

```
clave = "turno:bloqueado:med_2001:2026-05-20:10:30"
```

```
cliente_redis.set(clave, "pac_1001", ex=300)
```

```
print("Valor de la clave:", cliente_redis.get(clave))
```

```
print("Tiempo restante:", cliente_redis.ttl(clave), "segundos")
```

El parámetro `ex=300` indica que la clave expirará en 300 segundos, es decir, 5 minutos. Durante ese tiempo, la aplicación puede interpretar que el turno está bloqueado para otro paciente. Si el paciente no confirma la reserva y el tiempo se agota, Redis elimina la clave automáticamente.

Esta capacidad evita que una operación incompleta deje un turno bloqueado indefinidamente. También reduce la necesidad de procesos manuales de limpieza.

Bloqueo temporal de turnos y control de concurrencia

La concurrencia aparece cuando dos o más usuarios intentan realizar operaciones sobre el mismo recurso al mismo tiempo. En un sistema de turnos médicos, dos pacientes podrían intentar reservar el mismo horario con el mismo profesional. Si la aplicación no controla esa situación, podría registrar dos reservas para un único turno.

Redis permite implementar un bloqueo simple mediante la creación de una clave temporal. La idea es que la aplicación intente crear una clave de bloqueo solo si no existe previamente. En Redis, esto se logra con el parámetro `nx=True`, que significa “set if not exists”, es decir, crear la clave únicamente si todavía no existe.

```
import redis
```

```
cliente_redis = redis.Redis(
```

```

host="localhost",
port=6379,
decode_responses=True
)

clave_bloqueo = "turno:bloqueado:med_2001:2026-05-20:10:30"
paciente_id = "pac_1001"

bloqueo_creado = cliente_redis.set(
    clave_bloqueo,
    paciente_id,
    ex=300,
    nx=True
)

if bloqueo_creado:
    print("El turno fue bloqueado temporalmente para el paciente.")
else:
    print("El turno ya está siendo procesado por otro paciente.")

```

Si la clave no existe, Redis la crea y devuelve un resultado exitoso. Si la clave ya existe, no la reemplaza. De esta manera, se evita que dos pacientes bloqueen el mismo turno al mismo tiempo.

Este mecanismo no reemplaza todas las validaciones de una aplicación, pero ayuda a resolver un problema frecuente de concurrencia. En sistemas críticos, además del bloqueo temporal, deben existir validaciones al momento de confirmar la reserva en la base persistente.

Integración conceptual de MongoDB y Redis en el caso de turnos médicos

El caso de turnos médicos permite representar una arquitectura simple con dos responsabilidades diferenciadas. MongoDB almacena la información persistente del sistema, mientras que Redis administra estados temporales y rápidos.

MongoDB puede contener colecciones como pacientes, medicos, turnos y reservas. Cada colección almacena documentos relacionados con una entidad del dominio. Un

paciente tiene datos personales, un médico tiene especialidad, un turno tiene fecha y hora, una reserva vincula paciente, médico y horario.

Redis no necesita replicar toda esa estructura. Su función es más específica. Puede guardar claves temporales que representen bloqueos de turnos, intentos de reserva o estados de sesión. La información en Redis debe ser breve, clara y orientada a una consulta rápida.

Una posible arquitectura lógica sería:

Aplicación Python

```
|
|
|--- MongoDB: datos persistentes e historial
|
|
|--- Redis: bloqueos temporales y estados rápidos
```

La aplicación Python coordina el flujo. No se trata de que MongoDB “llame” a Redis ni de que Redis “actualice” MongoDB automáticamente. La aplicación es la responsable de decidir cuándo consultar cada base y cómo interpretar cada respuesta.

Flujo aplicado de reserva de un turno médico

Un flujo razonable de reserva puede organizarse en varias etapas. Primero, la aplicación recibe el identificador del paciente, del médico, la fecha y la hora solicitada. Luego consulta si el turno existe y si se encuentra disponible. Después intenta bloquear temporalmente el turno en Redis. Si el bloqueo falla, significa que otro usuario está procesando la misma reserva. Si el bloqueo se crea correctamente, la aplicación puede avanzar con la confirmación.

Al confirmar la reserva, la aplicación registra el documento correspondiente en MongoDB y actualiza el estado del turno. Una vez registrada la reserva, puede eliminar la clave temporal de Redis o dejar que expire automáticamente, según el diseño elegido.

Un script integrado puede representar este flujo de manera simplificada:

```
from pymongo import MongoClient

import redis

mongo = MongoClient("mongodb://localhost:27017/")
base = mongo["clinica_turnos"]
turnos = base["turnos"]
reservas = base["reservas"]
```



```
redis_client = redis.Redis(  
    host="localhost",  
    port=6379,  
    decode_responses=True  
)
```

```
paciente_id = "pac_1001"  
medico_id = "med_2001"  
fecha = "2026-05-20"  
hora = "10:30"
```

```
clave_bloqueo = f"turno:bloqueado:{medico_id}:{fecha}:{hora}"
```

```
turno = turnos.find_one({  
    "medico_id": medico_id,  
    "fecha": fecha,  
    "hora": hora,  
    "estado": "disponible"  
})
```

```
if not turno:
```

```
    print("El turno no existe o no está disponible.")
```

```
else:
```

```
    bloqueo = redis_client.set(  
        clave_bloqueo,  
        paciente_id,  
        ex=300,  
        nx=True
```

)

if not bloqueo:

```
print("El turno está siendo procesado por otro paciente.")
```

else:

```
reserva = {
    "paciente_id": paciente_id,
    "medico_id": medico_id,
    "fecha": fecha,
    "hora": hora,
    "estado": "confirmada"
}
```

```
reservas.insert_one(reserva)
```

```
turnos.update_one(
    {"_id": turno["_id"]},
    {"$set": {"estado": "reservado"}}
)
```

```
redis_client.delete(clave_bloqueo)
```

```
print("Reserva confirmada correctamente.")
```

El ejemplo muestra la integración mínima entre ambas bases. MongoDB se usa para verificar disponibilidad y registrar la reserva. Redis se usa para evitar que otro paciente tome el mismo turno durante el proceso. Python coordina la operación completa.

Este código no representa una aplicación hospitalaria completa. En una solución real habría autenticación, validaciones adicionales, manejo de errores, control de permisos, auditoría, logs, interfaz de usuario y posiblemente transacciones o mecanismos más robustos. Sin embargo, el ejemplo permite comprender el rol de cada componente dentro de un flujo aplicado.

Evaluación de resultados en operaciones de acceso a datos

Acceder a una base de datos no significa solamente ejecutar instrucciones. Una aplicación debe evaluar el resultado de cada operación. Esta evaluación permite saber si una inserción fue realizada, si una consulta encontró datos, si una actualización modificó documentos o si una operación fue rechazada.

En MongoDB, las operaciones devuelven objetos con información relevante. Una inserción devuelve el identificador generado. Una actualización informa cuántos documentos coincidieron y cuántos fueron modificados. Una eliminación informa cuántos documentos fueron borrados.

En Redis, muchas operaciones devuelven valores que indican éxito, existencia o ausencia. Por ejemplo, `set` con `nx=True` permite saber si la clave fue creada o si ya existía. `ttl` permite conocer el tiempo restante de una clave. `get` permite recuperar el valor asociado a una clave.

La evaluación de resultados es especialmente importante en procesos concurrentes. Si dos pacientes intentan reservar el mismo turno, no alcanza con consultar una vez MongoDB y asumir que el turno sigue libre. Entre la consulta y la confirmación puede intervenir otro usuario. Por eso Redis ayuda a incorporar una barrera temporal de control.

Un error frecuente es escribir código que ejecuta operaciones sin revisar sus resultados. Ese enfoque puede producir inconsistencias difíciles de detectar. Una aplicación bien diseñada debe validar cada paso crítico.

Manejo básico de errores

El acceso a datos puede fallar por múltiples razones: la base puede estar apagada, el puerto puede ser incorrecto, la cadena de conexión puede estar mal escrita, la red puede fallar o una operación puede recibir datos inválidos. Por eso, el código debe contemplar errores.

En Python, `try` y `except` permiten capturar excepciones. Una excepción es una situación anómala que interrumpe el flujo normal del programa.

```
from pymongo import MongoClient
```

```
from pymongo.errors import ServerSelectionTimeoutError
```

```
try:
```

```
    cliente = MongoClient(
        "mongodb://localhost:27017/",
        serverSelectionTimeoutMS=3000
```

```
)
cliente.server_info()

print("Conexión exitosa con MongoDB")

except ServerSelectionTimeoutError:

    print("No se pudo conectar con MongoDB. Verificar que el servicio esté activo.")
```

En este ejemplo, `serverSelectionTimeoutMS=3000` limita el tiempo de espera a 3000 milisegundos. Sin este límite, la aplicación podría quedar esperando más tiempo antes de informar el problema.

Para Redis, también se puede capturar un error de conexión:

```
import redis

try:

    cliente_redis = redis.Redis(

        host="localhost",

        port=6379,

        decode_responses=True

    )

    cliente_redis.ping()

    print("Conexión exitosa con Redis")
```

```
except redis.exceptions.ConnectionError:

    print("No se pudo conectar con Redis. Verificar que el contenedor o servicio esté activo.")
```

El método `ping` permite comprobar que Redis responde. Esta verificación es útil antes de ejecutar operaciones críticas.

Buenas prácticas en el diseño de acceso a datos

El acceso a bases NoSQL desde aplicaciones requiere decisiones de diseño. No alcanza con instalar librerías y ejecutar comandos. Es necesario definir responsabilidades, validar datos, controlar errores y evitar mezclas inadecuadas entre modelos.

Una buena práctica consiste en mantener separada la lógica de conexión y la lógica de negocio. La conexión define cómo llegar a la base; la lógica de negocio define qué operación corresponde realizar. En proyectos pequeños, ambas pueden aparecer en el mismo script por simplicidad, pero en sistemas más grandes conviene separarlas.

También es importante diseñar nombres de claves claros en Redis. Una clave como x1 no explica nada. Una clave como turno:bloqueado:med_2001:2026-05-20:10:30 expresa dominio, estado, médico, fecha y hora. Esa claridad facilita mantenimiento y diagnóstico.

En MongoDB, la flexibilidad documental no debe confundirse con desorden. Si una colección almacena turnos, sus documentos deberían mantener una estructura coherente. La ausencia de un esquema rígido no elimina la necesidad de diseño.

Otra práctica relevante es no guardar credenciales sensibles directamente en el código. En contextos académicos puede usarse una cadena local simple, pero en entornos reales deben emplearse variables de entorno, archivos de configuración seguros o servicios de gestión de secretos.

Finalmente, la aplicación debe verificar los resultados de las operaciones. Insertar, actualizar o bloquear datos sin revisar la respuesta puede generar errores silenciosos. La robustez de un sistema depende tanto de la operación ejecutada como de la validación posterior.

Errores frecuentes al integrar aplicaciones con bases NoSQL

Uno de los errores más habituales es pensar que NoSQL significa ausencia de diseño. MongoDB permite documentos flexibles, pero esa flexibilidad no debe llevar a estructuras improvisadas. Redis permite claves simples, pero una mala convención de nombres puede volver inmanejable el sistema.

Otro error frecuente es usar Redis como si fuera una base documental principal. Redis puede persistir datos según su configuración, pero su fortaleza habitual está en la velocidad, el trabajo en memoria y los estados temporales. Si se necesita historial, consulta por múltiples campos y trazabilidad de reservas, MongoDB resulta más adecuado.

También es común olvidar el control de concurrencia. Una aplicación puede consultar que un turno está disponible, pero esa información puede cambiar rápidamente si otro usuario actúa al mismo tiempo. Por eso, el bloqueo temporal con Redis ayuda a reducir riesgos en operaciones sensibles.

Un cuarto error consiste en no manejar fallas de conexión. Si MongoDB o Redis no están activos, la aplicación debe informar el problema de manera clara. Un mensaje genérico o una interrupción abrupta dificulta la identificación de la causa.

También debe evitarse la eliminación física indiscriminada de datos. En sistemas de salud, auditoría, banca o logística, muchas operaciones deben conservar trazabilidad. Cancelar un turno no necesariamente significa borrar la reserva; muchas veces significa cambiar su estado.

Criterios para decidir qué base utilizar

La elección entre MongoDB, Redis u otra base NoSQL debe responder al tipo de dato, al patrón de consulta, al volumen esperado, a la necesidad de persistencia y al tiempo de respuesta requerido.

MongoDB conviene cuando se necesitan documentos con estructura flexible, almacenamiento persistente, consultas por campos y representación de entidades con varios atributos. En el caso de turnos médicos, es adecuado para pacientes, médicos, turnos, reservas e historial.

Redis conviene cuando se necesitan respuestas muy rápidas, datos temporales, claves con vencimiento, contadores, sesiones, bloqueos o caché. En el caso de turnos médicos, es adecuado para bloquear un turno durante algunos minutos mientras se confirma la reserva.

SQL Server u otra base relacional puede ser conveniente cuando se requiere fuerte estructura tabular, integridad referencial estricta, transacciones complejas y consultas SQL tradicionales. Puede convivir con bases NoSQL en una arquitectura empresarial, pero no necesariamente debe reemplazarlas.

El criterio no debe ser elegir una tecnología por moda, sino asignar responsabilidades. Una arquitectura bien diseñada puede combinar modelos distintos siempre que cada uno tenga una función clara.

Síntesis conceptual

El acceso a estructuras NoSQL desde aplicaciones permite comprender cómo los datos se integran en procesos reales de software. Una base documental, una base clave-valor o cualquier otro motor NoSQL no funcionan de manera aislada dentro de una solución: son componentes que una aplicación utiliza para consultar, validar, registrar y responder operaciones.

Python permite representar de manera clara esta integración. Mediante librerías como pymongo y redis, una aplicación puede conectarse a MongoDB y Redis, ejecutar operaciones y evaluar resultados. MongoDB aporta persistencia documental para entidades e historial, mientras que Redis aporta velocidad y manejo de estados temporales mediante claves, TTL y bloqueos.

El caso de turnos médicos muestra la importancia de asignar responsabilidades. La reserva confirmada pertenece al mundo persistente y debe quedar registrada. El bloqueo momentáneo del turno pertenece al mundo temporal y debe expirar si la operación no se completa. La aplicación coordina ambos comportamientos y decide cómo actuar ante cada respuesta.

El diseño correcto no depende solo de conocer comandos. Requiere comprender la naturaleza del dato, el flujo de negocio, los riesgos de concurrencia, la necesidad de trazabilidad y la evaluación de resultados. Acceder a datos NoSQL desde aplicaciones implica integrar tecnología, lógica y criterio de diseño para construir soluciones consistentes, escalables y comprensibles.